

University Of Asia Pacific
Department Of CSE

Lab Report - 4

Course Code: CSE-402

Course Title: Operating System Lab.

Submitted By:

Name: Tashnimul Hasan Shifat

Student Id: 23101142

Semester: 4th Year 1st Semester

Section: C - 2

Date: 28 - 08 - 2026

Submitted To

Atia Rahman Orthi

Lecturer

UAP, CSE

1. Problem:

Given the following processes with their **Arrival Time (AT)** and **Burst Time (BT)**:

Process	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Implement the **Preemptive Shortest Job First (PSJF)** CPU Scheduling Algorithm using a **Time Quantum (TQ) of 2 units**.

Calculate the following for each process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes.

2. Source Code:

gg > ...

```python
processes = [
    ["P1", 4, 2],
    ["P2", 2, 2],
    ["P3", 1, 3],
    ["P4", 0, 6],
    ["P5", 3, 1]
]

quantum = 2
n = len(processes)


remaining = {}

for p in processes:
    remaining[p[0]] = p[2]

CT = {}
sequence = []
time = 0

# Scheduling
while len(CT) < n:

    available = []

    for p in processes:
        name = p[0]
        AT = p[1]

        if AT <= time and remaining[name] > 0:
            available.append(p)

```

```
34
35      if len(available) == 0:
36          time += 1
37          continue
38
39
40      current = min(
41          available,
42          key=lambda p: remaining[p[0]]
43      )
44
45      name = current[0]
46
47
48      sequence.append(name)
49
50
51      run_time = min(quantum, remaining[name])
52
53      remaining[name] -= run_time
54      time += run_time
55
56
57      if remaining[name] == 0:
58          CT[name] = time
59
60
61
62  TAT = {}
63  WT = {}
64
65  for p in processes:
```

```
66
67      name = p[0]
68      AT = p[1]
69      BT = p[2]
70
71      TAT[name] = CT[name] - AT
72      WT[name] = TAT[name] - BT
73
74
75
76
77  print("Process Execution Sequence:")
78  print(" -> ".join(sequence))
79
80
81
82
83  print("\n")
84  print(f"{'P':<8}{'AT':<8}{'BT':<8}{'CT':<8}{'TAT':<8}{'WT':<8}")
85  print("-" * 48)
86
87  total_tat = 0
88  total_wt = 0
89
90  for p in processes:
91
92      name = p[0]
93      AT = p[1]
94      BT = p[2]
95
96      print(
97          f"{name:<8}"
```

```
        print(
            f"{name:<8}"
            f"{AT:<8}"
            f"{BT:<8}"
            f"{CT[name]:<8}"
            f"{TAT[name]:<8}"
            f"{WT[name]:<8}"
        )


        total_tat += TAT[name]
        total_wt += WT[name]



    avg_tat = total_tat / n
    avg_wt = total_wt / n

    print("-" * 48)
    print(f"Average TAT = {avg_tat:.2f} sec")
    print(f"Average WT  = {avg_wt:.2f} sec")
```

3. Output:



4. Discussion:

In this experiment, five identical processes were used to implement both Preemptive SJF and Non-Preemptive SJF CPU scheduling algorithms, and their performance was compared based on Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, and Average WT. In Preemptive SJF, a running process can be stopped if a newly arrived process has a shorter remaining execution time, which allows shorter processes to get CPU time earlier and finish faster. This algorithm produced an Average TAT of 6.0 and an Average WT of 3.2. On the other hand, in Non-Preemptive SJF, once a process starts, it cannot be interrupted until it finishes, so even if a shorter process arrives, it must wait, resulting in a higher

Average TAT of 7.4 and Average WT of 4.6 compared to Preemptive SJF. From these results, Preemptive SJF has both lower Average TAT and lower Average WT, meaning that for this given set of processes, Preemptive SJF performed better overall than Non-Preemptive SJF and proved to be the more efficient scheduling algorithm.

5. Conclusion:

This experiment successfully implemented and compared Preemptive SJF and Non-Preemptive SJF scheduling algorithms, and the results showed that Preemptive SJF performed better for the given set of processes, achieving an Average TAT of 6.0 and an Average WT of 3.2, whereas Non-Preemptive SJF resulted in a higher Average TAT of 7.4 and an Average WT of 4.6. Based on these lower values of Average Turnaround Time and Average Waiting Time, it can be concluded that Preemptive SJF is the more efficient scheduling algorithm for this dataset. However, Non-Preemptive SJF has the advantage of being simpler to implement, since once a process begins execution, it continues running until completion without requiring any interruption or context switching.